PALADIN: Policy-Aware Layered Agentic Decision Intelligence over the Model Context Protocol

Anonymous Author(s)

Abstract

Autonomous LLM agents now invoke external tools through the Model Context Protocol (MCP), turning every tool call into an authorisation decision the protocol itself does not make. We argue this is an *access-control* problem—and that the field has been answering the wrong question: *semantic correctness is not authorisation correctness*. The tools that best accomplish a task can be exactly the ones an agent is not permitted to use, while a poorly-chosen bundle may be perfectly admissible. We introduce **PALADIN**, a deterministic invocation-time authorisation pipeline (SPIFFE/SPIRE identity, RBAC, ABAC, then TRAC), whose novel stage is **TRAC**—a *task-relational* admission primitive that authorises an invocation on the coherence between the task and its proposed action-set.

Our central result: this deterministic floor holds worst-case security-failure rate to a fixed 10.3%, identical across every model. On ASTRA [11], an agentic tool-selection safety benchmark, six LLMs from three vendors used *alone* as admission controllers wrongly admit 27–67% of unsafe tool calls—unreliable, with no monotonic improvement across newer or stronger models. Composed *behind* the floor, the *same* models all converge to at most 7.3%. Around this we build a measurement framework for layered authorisation: it separates an LLM *judging* a candidate bundle from one *generating* it, and shows that naïve attribution misranks which policy layers matter. The floor guards *semi-honest* mis-scoping (the common case) at a bounded false-block cost; it composes with prompt-injection defences for adversarial input and needs no bespoke runtime (verified OPA/Rego parity). All policies, logs, and scripts are released.

CCS Concepts

• **Security and privacy** → **Access control**; *Authorization*; Logic and verification; • **Computing methodologies** → *Multi-agent systems*.

Keywords

access control, RBAC, ABAC, typed predicates, Model Context Protocol, agentic AI, LLM safety, policy enforcement, zero trust

ACM Reference Format:

Anonymous Author(s). 2027. PALADIN: Policy-Aware Layered Agentic Decision Intelligence over the Model Context Protocol. In *Proceedings of The 32nd ACM Symposium on Access Control Models and Technologies (SACMAT '27)*. ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

SACMAT '27, To be announced

ACM ISBN 978-1-4503-XXXX-X/27/06
https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

1.1 Motivation

LLM agents that autonomously select and invoke external tools have proliferated across enterprise and consumer domains [43, 45], building on tool-augmented language models [36] and ReAct-style reasoning-and-acting loops [47]. They now standardise on the Model Context Protocol (MCP) [1] as the de facto tool-invocation interface. MCP deployments typically assume a trusted execution environment with no native identity verification, permission scoping, or deterministic predicate evaluation. The security gap is consequential: an agent tasked with "investigating anomalous metrics" might autonomously select tools that modify production alert rules, mutate database state, or initiate financial transactions far outside its intended mission. Recent analyses catalogue 16 MCP-lifecycle threat categories [16] and document exploits including credential theft, tool poisoning, and remote code execution [21].

Existing LLM-security work addresses fragments. Prompt-injection defences [25, 49] target adversarial reasoning manipulation; output filters such as Llama Guard [19] constrain generation content; agent-safety benchmarks [48, 50] evaluate behaviour in controlled environments. None reasons about the security properties of *individual tool invocations* within a multi-domain operational context. Chu et al. [6] identify a layered attack-surface model but provide no enforcement mechanism; we instantiate one as a staged invocation-time pipeline.

This exposes a reframing the agentic-AC literature has largely missed: *task-tool semantic correctness is not authorisation correctness*. A bundle a task-tool benchmark scores as "correct"—the tools that best accomplish the task—may still be *inadmissible* under the agent's role, attribute, and task-coherence constraints; conversely a benchmark-incorrect bundle may be perfectly admissible. Authorising agentic tool use is therefore an access-control problem, not a retrieval-accuracy one. It needs an enforcement primitive that conditions on the task↔bundle relation—a relation outside the *conventional* attribute vocabulary of RBAC and ABAC, yet, like ReBAC and Purpose-BAC, enforceable on a standard policy engine once derived (§5.4.3).

1.2 Contributions

Our primary contribution is an *access-control* one: we recast agentic tool invocation as an authorisation problem (semantic correctness ≠ authorisation correctness) and introduce a *task-relational* admission primitive—**TRAC** (task-relational access control)—that authorises on the discriminating signal conventional access control omits: the coherence between the task and its proposed action-set. We do *not* claim a new formal model: TRAC needs no new rule-engine

machinery and runs as rules-as-data on a commodity engine (verified OPA/Rego parity, §5.4.3). The novelty is the *reframing*, the *leak-free derivation* of that coherence attribute at invocation time (consuming no ground-truth label, §4.6), a *measurement methodology* for layered authorisation (how a deterministic policy stack and LLM judgement compose), and the empirical demonstration that measured semantic coherence is a reliable, load-bearing authorisation attribute—all realised in **PALADIN** (Policy-Aware Layered Agentic Decision Intelligence), our deterministic invocation-time authorisation pipeline (SPIFFE/SPIRE identity → RBAC → ABAC → a fact transformer → TRAC) in which TRAC is the novel stage. Concretely:

(1) A **methodology for measuring layered authorisation**: a dual-mode validation/selection protocol (the LLM *judging* a candidate bundle vs. *generating* one) that most benchmarks conflate; two metric conventions (security- vs. permissivity-view) that disagree on layer importance and must be reported jointly; paired task-level bootstrap CIs on layer marginals; and a dataset-ceiling analysis bounding exact-match at 30–40% (§4, 4.5, 5.4.4).
(2) A **task-relational admission primitive, TRAC**: it authorises an invocation on the *measured coherence between the task and the proposed bundle*—in-domain coverage, mutation safety, task–tool relevance—a task↔bundle relation not provisioned as a conventional RBAC/ABAC attribute and distinct from the workflow (TBAC [41]), entity (ReBAC [15]), and *declared*-purpose (Purpose-BAC [5]) axes. Derived by a fact transformer, it consumes *no ground-truth label* (BM25 domain inference, §4.6) and is enforceable on commodity engines (OPA/Rego parity, §5.4.3); we claim an empirical, load-bearing authorisation signal, not a new formal model. Every decision recomputes from the released logs without LLM calls.
(3) A **deterministic security floor whose *magnitude* we measure**: conditioning on the bundle not the model, the stack returns a bit-identical profile (security-failure 10.3%, precision/security 0.63, recall/availability 0.45) for every model—the empirical content is the floor's magnitude, not the by-construction invariance. Used *alone*, the same models leak 27–67% on the same inputs; composing the model's verdict behind the floor sharpens security-failure to 2.1% (gpt-4o) and never exceeds the floor. A blind cross-vendor panel (Claude Opus 4.8, Sonnet 4.6, Gemini 2.5 Pro; all 6,942 rows) converges to security-failure $\leq 7.3\%$ (§5.3.1, 5.2.3).
(4) **Complementary layers, measured two ways** (paired CIs): drop-one marginals are −20.7pp (RBAC) and −12.0pp (TRAC)—each individually load-bearing—while ABAC's is only −0.9pp (it overlaps RBAC) yet load-bearing when RBAC is absent (successive −23.4pp). Under a label defined independently of every policy artefact TRAC's marginal becomes the *largest* (§5.3.3); the successive/drop-one gap exposes a first-firing attribution pitfall (§5.4.1).
(5) **Attribution paradox and corroborated coverage**: (a) first-firing denial attribution misleads—RBAC claims 82% of denials, yet ABAC (claiming 4%) carries the *largest* successive marginal (−23.4pp)—a pitfall in much of the agent-safety literature; and (b) *corroborated coverage* recovers legitimate work at near-zero security cost. We also reproduce ASTRA's LLM-ResM [11] under a stricter per-bundle protocol (§5.4.1, 5.4.2).

What this paper is not. PALADIN is a security-first invocation-time floor, not an end-to-end agent-security solution: prompt-injection, tool-output exfiltration, MCP supply-chain, and side-channel attacks are out of scope and compose with it (§6.1).

2 Related Work

2.1 Access Control, Policy Languages, and Task-Tool Benchmarks

Five decades of access-control foundations span lattice models [4], RBAC [13, 35], ABAC [17], and policy languages with decidable analysis (Cedar [9], XACML [27], Margrave [14]). These were designed for human principals on static boundaries; agents construct invocations dynamically via probabilistic reasoning and span multiple domains per task, producing tool bundles whose coherence with the task carries security-relevant signal.

Two AC lineages model structure beyond the subject and are the natural reference points for a "task-relational" claim. *Task-Based Access Control* (TBAC) [41] ties permission *activation* to a task's workflow lifecycle—permissions are granted and revoked just-in-time as a task advances through its steps. *Relationship-Based Access Control* (ReBAC) [15] authorises on *relationships among entities* (owner-of, member-of, . . .). Our task-relational predicate is orthogonal to both: it gates on the *semantic coherence between the task's intent and the proposed tool bundle*—is this bundle in-domain and sufficient for what the task asks?—not on a workflow position (TBAC) or an entity graph (ReBAC). It is complementary, not a replacement: an agent could be scoped by TBAC over a workflow *and* checked by a task-relational coverage predicate at each invocation. Two further lineages sharpen the comparison: *Purpose-Based Access Control* [5] authorises by the *purpose* of an access—one the requester *states* and the system validates only for *entitlement to claim* (via conditional roles over role and system attributes), never for truthfulness—and *Usage Control* (UCON) [28] unifies authorisations, obligations, and conditions over mutable attributes with continuity of enforcement. TRAC differs in *what* it conditions on: a *measured* coherence between the task text and the proposed bundle, computed at invocation time from public tool descriptions—not a self-declared purpose label (which the semi-honest agent controls) nor an obligation model—and it composes with, rather than replaces, either.

Relative to Cedar and HashiCorp Sentinel—the substrates most often proposed for agentic authorisation—TRAC adds (i) typed predicates over *derived* continuous relevance and coverage scores—BM25 task–domain and tool–task relevance over public metadata—with thresholds revisable as policy, (ii) a *task-relational* coverage predicate—whether the proposed bundle covers the inferred task domain—conditioning on the task↔bundle relation rather than on caller or resource attributes alone, and (iii) bundle-level decision points composing over the full emitted tool set. We *do* provide a head-to-head OPA/Rego re-implementation: the RBAC, ABAC, and

TRAC layers run as rules-as-data on a standard Open Policy Agent runtime with decisions verified bit-identical to our reference engine (§5.4.3). This confirms our novelty is the task-relational *predicate* and the measurement framework, not the rule engine—which is substrate-portable.

Could one of these methods be used *instead* of the task-relational predicate? Not on this benchmark. The discriminating signal is the task↔bundle coherence, which lies outside the native vocabulary of every lineage above (subject, attribute, workflow, relationship, purpose, obligation). The policy *engines*—Cedar, XACML, OPA/Rego, Sentinel (Margrave adds offline policy verification)—can *host* such a predicate (we verify OPA/Rego decision parity, §5.4.3) but ship none themselves. So reusing any of them *without* adding a measured coverage predicate leaves that signal unenforced and lands at the RBAC∧ABAC operating point—security-failure 0.223 versus the full stack's 0.103, a 2.2× gap (Table 12) whose +12pp is precisely the predicate's marginal—so they are *substrates or complements* for the predicate, not substitutes.

The closest task-tool benchmark is ASTRA [11], whose LLM reasoning matcher (LLM-ResM, per-tool GPT-4o with AND-aggregation) reports $F_1 \approx 0.67$ on the $N = 3$ multi-tool test split. We re-use ASTRA's corpus and labels, reproduce LLM-ResM under a stricter per-bundle protocol (§5.2.2), and evaluate what a deterministic stack composed on top achieves on the same inputs.

Concurrent with our work, the ASTRA team have extended their pipeline into a hybrid runtime framework, CASA [12], that pairs five *deterministic* message-integrity checks (tool-definition, request-authorisation, action-alignment, parameter, and result-fidelity verification) with a two-stage *semantic* layer—an LLM extracts the task from a multi-turn conversation, then an LLM judges each requested tool's relevance. This contemporaneous effort sharpens rather than pre-empts our contribution on three axes. First, their coherence signal is an *LLM* per-tool judgment, whereas TRAC is a *deterministic, leak-free* attribute derived from public metadata (§4.6) and enforced as ordinary rules on an ABAC-class engine (§5.4.3)—exactly the recomputable floor their own conclusion motivates when it observes that "current semantic checks are not yet sufficient on their own" for high-stakes tool use. Second, their deterministic checks govern *message-flow integrity* (tampering, tool swaps, result falsification), which is orthogonal to—and composable with—our deterministic capability-*coherence* envelope. Third, and most directly, CASA matches *each tool independently* and explicitly lists bundle-level coherence—relevance that "depends on the combination rather than individual tools"—as an *open problem*; that combination is the set-valued task↔bundle relation TRAC gates (§3.5; bundle D). Their genuine advance over prior work—multi-turn task *extraction* from conversational context—is complementary to, and outside the scope of, our single-task, persona-grounded setting; and unlike their extractor, which trusts the recovered task, we assume a *semi-honest* agent whose declared purpose cannot be trusted (§3.2).

2.2 Zero-Trust, Agent Identity, and MCP Security

NIST Zero Trust [33] and SPIFFE [40] establish cryptographic workload identity but have not been applied to AI-agent identity; recent delegation work [30, 39] and defence-in-depth surveys [6, 22, 26] map the layered attack surface without providing formal enforcement.

The agent-safety community has produced evaluation frameworks—Agent-SafetyBench [50], ToolEmu [34], GuardAgent [46], R-Judge [48], ALMITA [2], Constitutional AI [3], deceptive-alignment analyses [18]—and prompt-injection defences [10, 19, 23, 25, 49]. These treat the agent monolithically and do not enforce authorisation at the invocation boundary. MCP-specific work has begun mapping the landscape (16 threat categories [16], practical exploits [21], cross-server tool poisoning [20]); multi-agent and behavioural-trust efforts [7, 8, 29, 31, 37, 44, 51] establish the problem space but offer ad-hoc rather than staged enforcement. Foundational tool-augmented architectures [36, 42, 43, 45, 47] provide the agents but no governance for their invocations; PALADIN is architecture-agnostic and composes on top.

3 The PALADIN Pipeline

PALADIN authorises each tool invocation through an ordered stack of deterministic gates that culminates in TRAC. We first fix the admission model and its short-circuit algebra (§3.1), state the threat model (§3.2), give the design rationale and walk a single request through the pipeline (§3.3), then develop the conventional layers (§3.4) and, as the novel stage, TRAC (§3.5).

3.1 Gate Control and Short-Circuit Composition

A tool invocation request is modelled as a 5-tuple

$$r = \langle \iota, \tau, T, M, \delta \rangle \tag{1}$$

where ι is the agent identity (SPIFFE ID), τ the task description, $T = \{t_1, \ldots, t_n\}$ and $M = \{m_1, \ldots, m_n\}$ the selected tools and their MCP servers, and δ the environmental context (timestamp, compliance tier).

Unit of decision: a per-invocation reference monitor. Π is a reference monitor at the MCP invocation boundary: it authorises *each request* r at the moment the agent emits it, as an operating system mediates each system call—not a single up-front verdict over a whole task. A request carries the task τ as trusted provenance (§3.2) together with the tool(s) invoked at that step—one tool in the common case, or the small set an agent emits in a single step (what the evaluation corpus supplies per task, our unit of evaluation). The agent never declares its whole task in advance and Π never enumerates future requests: because τ is present at *every* step, S_6 can judge whether *this* step coheres with the mission without foreknowledge of later ones. Admission is therefore incremental—an agent whose in-domain reads are admitted keeps making progress while a single out-of-scope call (a cross-domain write, say) is denied, after which it re-plans and completes the task through admitted calls. Stages S_1–S_4 evaluate the classical subject/object/operation triple over the tool invocation $(\iota, (t, m), a(t))$; S_6 adds one object conventional access control does not name—the capability the bundle exercises *in service of* τ (§3.4). What Π mediates is that invocation boundary—which capability an agent may exercise—not the internal objects of the remote service, whose own access control composes with,

and is out of scope for, this layer. §3.6 traces a concrete request end-to-end.

The short-circuit composition operator $\triangleright$ between stages is

$$(S_i \triangleright S_j)(r) = \begin{cases} \text{DENY}_i, & \text{if } S_i(r) = \text{DENY}, \\ S_j(r), & \text{otherwise.} \end{cases} \quad (2)$$

S_5 is not a decision stage but a deterministic *fact-transformer* $\phi : r \mapsto \Phi(r)$ that materialises the typed fact base consumed by S_6; it cannot return DENY. The full pipeline is therefore

$$\Pi(r) = \big(S_1 \triangleright S_2 \triangleright S_3 \triangleright S_4 \triangleright S_6\big)\big(r, \phi(r)\big), \quad (3)$$

i.e. ϕ runs once when S_1–S_4 all admit, and S_6's predicates evaluate over $\Phi(r)$.

Each stage records `ALLOW`, `DENY`, or `NOT_EVALUATED`; the final decision is the conjunction of evaluated stages. A `DecisionResult` carries the per-stage map, final decision (`ALLOW`/`DENY`), denial source, and audit trace. Short-circuiting yields three benefits: computational efficiency (S6 is skipped when cheaper checks suffice), security hardening (later stages cannot mask upstream denials), and audit clarity (the denial source unambiguously identifies the triggering concern).

Beyond the per-stage states, the pipeline returns a single binary admission decision: $\Pi(r) = \text{ALLOW}$ only if every evaluated stage allows, and DENY otherwise. There is no third outcome—a denied high-risk write simply stays `DENY`—which keeps every layer expressible as a standard ALLOW/DENY policy (§5.4.3). Advisory TRAC rules (§3.5) may additionally raise non-blocking alerts on an admitted bundle, but these never change the admission decision and are excluded from the security metrics.

The operator $\triangleright$ gives Π three properties. **(P1) Monotonicity:** adding a stage can only convert ALLOW to DENY (immediate from the short-circuit semantics). **(P2) Order-independence of unique necessity:** if a single S_k is the only stage that would deny r, $\Pi(r) = \text{DENY}$ regardless of ordering. **(P3) Fail-secure default:** predicates default conservatively on missing facts and stage exceptions are wrapped to DENY, so partial failure cannot silently degrade to ALLOW. P1 grounds the Δ-value reading; P2 underpins the attribution analysis (§5.4.1).

3.2 Threat Model and Deployment Envelope

Adversary. A *semi-honest agent*: an authenticated LLM-driven workload that, given an operator- or workflow-supplied task, may select and emit a tool bundle that exceeds scope, crosses domains, targets writes without prior reads, or otherwise violates policy—through hallucination or over-helpful, helpfulness-optimised behaviour. It controls the *emitted bundle*; the task text is trusted provenance (operator/workflow), which is what makes the task-relational predicates meaningful. Active manipulation of the task text itself—prompt injection *into the task*, crafted to steer the relevance predicates—is out of scope and composes with the prompt-injection defences of §6.1.

Trust assumptions. We trust the SPIFFE/SPIRE identity substrate *and the mutual-TLS transport it anchors* (§3.4), policy YAML/JSON integrity at load, the MCP tool registry, and the deterministic pipeline; we do *not* trust LLM-emitted bundles or natural-language justifications, and we admit *no* self-reported model confidence as a policy input—every score TRAC thresholds is recomputed deterministically from public metadata and the trusted task text (§4.6).

Table 1: PALADIN's design principles, the mechanism realising each, and why it is needed. Several are consolidated from properties stated where they are first used (short-circuit benefits, §3.1; fail-secure and monotonicity, §3.1; substrate portability, §5.4.3). The optional LLM verdict gate (G) is listed as a first-class layer, confined so it can only add denials.

Principle	Mechanism	Why
Zero-trust identity	S_1 SPIFFE/SPIRE + mTLS	MCP assumes a trusted host; verify the agent and the channel, do not trust the caller
Closed-world tools	S_2 registry	admit only registered tools; unknown/poisoned servers cannot be invoked
Least privilege (role)	S_3 RBAC	an agent gets only the (domain, tool) pairs its persona needs
Context-sensitive gates	S_4 ABAC	clearance/trust/department/risk conditions a role cannot express
Leak-free, replayable facts	S_5 transformer ϕ	decisions recompute from logs with no LLM call and no ground-truth label
Task-relational least privilege	S_6 TRAC	authorised $\neq$ coherent: catch hallucinated/over-helpful bundles S_1–S_4 admit
Optional model second-opinion	G LLM verdict (validation)	a conjunctive gate behind the floor: can only *add* denials, never admit what the floor denies
Fail-secure default	all stages (P3)	missing facts and stage exceptions resolve to DENY
Cheap-to-costly ordering	short-circuit $\triangleright$	set-membership first, fact-derivation last: efficiency, no masking, audit clarity
Substrate portability	rules-as-data	commodity engines (OPA/Rego parity) reproduce every decision

In-scope. Within-domain mis-scoping, cross-domain capability leakage, mutation-without-read, hard-capability gaps, and trust-state misalignments (operationalised by the evaluation dataset's `wrong`/`null` negatives). **Out of scope:** policy-file compromise, side channels, DoS, MCP supply-chain attacks, active adversarial prompts crafted to evade specific predicates, and tool-output exfiltration. **Security objective:** minimise the security-failure rate (Eq. 9) subject to a bounded false-block rate on valid requests; the trade-off point is set by policy, not architecture.

Deployment envelope. PALADIN is an invocation-time authorisation layer, not an end-to-end agent-security solution. The exclusions above are not minor: each is an active research area, and production deployment composes PALADIN with output filters (e.g., Llama Guard [19]), signed registries, network-layer rate limiting, and prompt-injection defences [25]. Contribution claims are scoped to that envelope. TRAC's two enforcing predicates are deterministic relevance controls over the task text, so its deterministic guarantee targets *semi-honest* mis-scoping—hallucination, over-helpfulness, and benign scoping errors (the evaluation dataset's non-adversarial `wrong`/`null` negatives), the dominant failure mode of today's agents. Active task-text manipulation—an adversary crafting wording to lexically match a target bundle—is owned by trusted task provenance and the composed prompt-injection controls above, not by this layer.

3.3 Design Rationale and Pipeline Overview

PALADIN is assembled from a small set of security principles, each realised by one mechanism in the stack (Table 1). The design is deliberately *layered and deterministic*: independent concerns are separated into stages that each admit or deny for one auditable reason, and the only non-deterministic decider—an optional LLM verdict—is confined so that it can never weaken the guarantee.

Why this order. Stages are ordered by increasing cost and decreasing certainty. The cheapest, most certain checks—cryptographic identity and registry membership—run first; role and attribute policy follow; the costliest signal, the leak-free relevance-and-domain fact derivation that S_6 consumes, runs *only* once every cheaper stage has admitted (Eq. 3). Short-circuit composition then guarantees that a later, more permissive-looking

stage can never overturn an upstream denial (monotonicity, P1, §3.1) and that the denial source is unambiguous for audit (§3.1).

Why a deterministic floor, then an optional model verdict. The deterministic stages are model-independent and recomputable from the released logs; the LLM verdict, when present (validation mode), composes *conjunctively* behind them (FULL = floor ∧ verdict), so it can only *add* denials, never admit what the floor denies. This bounds the trusted computing base to the deterministic stack and keeps the model out of the decision's integrity: a compromised or hallucinating verdict degrades permissiveness, not security (§3.2).

Why facts precede rules. Separating fact derivation ($\phi = S_5$) from predicate evaluation (S_6) makes every task-relational decision a pure function of typed facts. The same $\Phi(r)$ yields the same, individually explained verdict on replay, and the rules—being data over derived attributes—port unchanged to a commodity policy engine (§5.4.3).

Pipeline realisation. The Unified Decision Engine executes these principles as six sequential stages with short-circuit semantics: any DENY from stages S1–S4 terminates the pipeline immediately (Figure 1). S5 extracts typed facts; S6 (TRAC) consumes them for predicate-level evaluation; in *validation* mode an optional LLM verdict gate G then composes conjunctively behind S_6.

Two evaluation modes. We exercise this pipeline in two complementary modes that differ *only* in where the proposed tool bundle originates; the six deterministic stages are identical in both (full protocol in §4.3). In *validation* mode the LLM is handed the task *and* a candidate bundle and returns the admit/deny *verdict* G of Figure 1. In *selection* mode the LLM is given the task *alone* and must *generate* the bundle, which is then routed through the *same* deterministic stages with no verdict gate. The two modes isolate the two ways an LLM can fail an authorisation—mis-*judging* a bundle it is shown versus mis-*selecting* the bundle it emits—and let us show that the same deterministic floor recovers both (§5.3).

Table 2 names the classical access-control triple—subject, object, operation—that each *deciding* stage evaluates, with its admission condition, and adds the LLM as its one non-deterministic component. We list only components that render an access-control verdict: authentication (S_1) *establishes* the subject rather than authorising an access, and the fact transformer (S_5) materialises $\Phi(r)$ without deciding, so neither is a row (both are detailed in §3.4). S_2–S_4 are a standard reference monitor over the tool invocation; ABAC (S_4) reuses the *same* subject and object as RBAC and merely adds attributes to the policy. S_6 is the one *deterministic* stage whose object—the capability the bundle exercises in service of the task—is not an attribute RBAC or ABAC *provisions*, but a relation over the task and the whole bundle that must be *derived* (§3.5); the LLM's role is mode-dependent—in validation it is the gate G, an *optional* second opinion behind S_6; in selection the same model instead *generates* the bundle the deterministic stages decide.

3.4 Conventional Layers: Identity, Registration, RBAC, and ABAC

This section derives the typed predicates each layer consumes—from persona-scoped RBAC grants and attribute rules, through

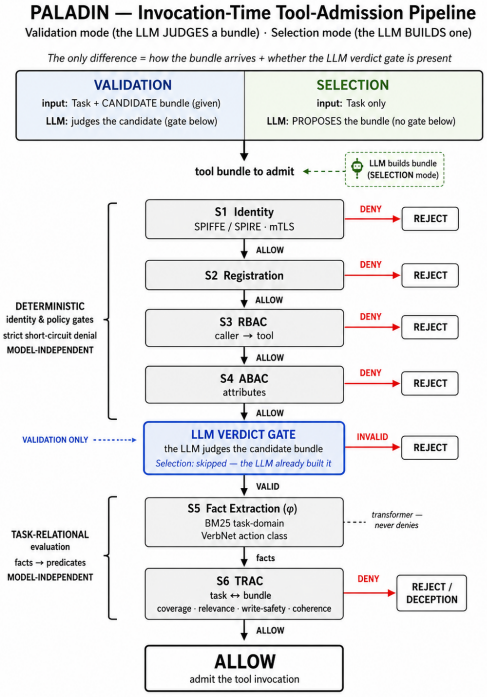

Figure 1: PALADIN's six-stage admission pipeline (short-circuit S1–S4; S5 fact transformer → S6 TRAC). The deterministic stages are model-independent; the LLM verdict gate composes *conjunctively* with the deterministic floor (its drawn position is presentational—FULL = floor ∧ verdict) and is present only in validation, while in selection the LLM instead generates the bundle.

Table 2: The subject, object, and operation each *deciding* stage evaluates, with its admission condition. Only genuine access-control stages appear: authentication (S_1) establishes the subject, and the fact transformer (S_5, ϕ) renders no verdict, so neither is a row (both in §3.4). S_2–S_4 form a classical reference monitor over the invocation $(\iota, (t, m), a(t))$; ABAC (S_4) uses the *same* subject and object as RBAC, adding subject-, resource-, action-, and environment-*attributes* to the policy. S_6's object—the capability the bundle exercises in service of τ—is not an attribute RBAC or ABAC *provisions* but a *derived* relation over τ and the whole bundle (§3.5). The LLM (last row) has a *mode-dependent* role: in *validation* it is the gate G—the one *non-deterministic* decider—composing conjunctively behind S_6 (FULL = floor ∧ verdict) so it can only add denials; in *selection* the *same* model instead *generates* the object (the candidate bundle) that S_2–S_6 decide, rendering no verdict of its own.

Stage	Subject	Object	Operation	Admits iff
S_2 Registration	agent	tool@server (t, m)	invoke	every (t, m) registered
S_3 RBAC	role $\rho(\iota)$	(domain, tool)	invoke as $a(t)$	$(m, t) \in \rho(\iota)$
S_4 ABAC	agent *(+attrs)*	resource *(+attrs)*	read/write/destructive	subject/resource/env attributes permit
S_6 TRAC	agent	capability set for τ	exercise bundle for τ	bundle coheres with τ
G LLM *(val./sel.)*	agent	bundle for τ	judge *(val.)* / generate *(sel.)*	*val.*: bundle consistent with τ; *sel.*: model emits it

the read/write action classes inferred from tool metadata, to the task-relational facts TRAC evaluates.

S1–S2 are deterministic set-membership checks: S_1 admits iff $\iota \in \mathcal{I}_{\text{valid}}$, and S_2 admits iff every $t \in T$ is in the registry $\mathcal{R}(m_t)$. **S3 (RBAC)** evaluates role-based permissions:

$$S_3(r) = \begin{cases} \text{ALLOW} & \text{if } \forall (m_j, t_j) \in \text{zip}(M, T) : (m_j, t_j) \in \rho(\iota) \\ \text{DENY} & \text{otherwise} \end{cases} \tag{4}$$

where $\rho(\iota)$ maps an identity to its permitted (domain, tool) pairs. **S4 (ABAC)** implements first-match semantics over attribute conditions,

$$S_4(r) = \text{first-match}(\mathcal{P}_{\text{ABAC}}, \text{attrs}(r)), \tag{5}$$

with $\mathcal{P}_{\text{ABAC}}$ the ordered policy set and $\text{attrs}(r)$ extracting subject, resource, action, and environment attributes. **S5** is the fact-transformer ϕ of Eq. 3: it derives typed facts without making any decision.

Agent-adapted RBAC maps agent personas to permissions over tool bundles across MCP domains, with three adaptations from traditional RBAC: tool-level granularity (per individual tool rather than per service endpoint); wildcard domain grants for trusted agents (e.g., `grafana.*`); and cross-domain permission intersection requiring permissions in *all* referenced domains across dynamically-constructed multi-tool bundles.

ABAC first-matches an ordered rule set over the subject, resource, action, and environment attributes of the request (§3.4). The released ruleset contains 6 rules in three families, conditioning on *subject* (department, clearance level, trust score), *resource* (compliance tier, data sensitivity, risk level), and *action* (write, destructive-write) attributes: *department/compliance isolation* (Financial/PCI-DSS resources restricted to Finance; Infrastructure writes to Engineering or Infrastructure), *clearance gates* (L2 to read high-risk resources, L3 to write them, L3 for destructive operations), and a *trust gate* (trust score ≥ 0.8 to write). A rule denies when all its conditions hold; conditions fail closed on an absent attribute; the request is denied if any rule fires.

Action classification. Each tool's action class $a(t) \in \{\text{read}, \text{write}\}$ (`destructive` a write subtype) is derived deterministically from the two universally-available MCP fields—tool *name* and *description*—by a verb lexicon grounded in established verb classes (Levin's verb classes [24], catalogued in VerbNet [38]) mapped to a read/write/destructive taxonomy (CRUD; RFC 9110 safe/unsafe methods). The leading description verb is lemmatised and class-mapped, ambiguous verbs are read-guarded (*run/execute a query* → read), and the tool name escalates only toward write/destructive (fail-safe defaults). There is no curated per-tool label map. Request-level predicates follow by composition—ContainsWrite $= \bigvee_{t \in T}(a(t) = \texttt{write})$, and analogously ContainsDelete and ContainsRead—and become typed inputs to TRAC.

Agnostic capability. The capability a tool provides is the pair $\{d(t)\}$:$\{a(t)\}$ of §3.5—its MCP domain $d(t)$ and the action class $a(t)$ above—with the subsumption $\{d\}$:write $\sqsupseteq$ $\{d\}$:read. There is *no* capability catalog, entailment ontology, or hard/soft capability tier: coverage is the single domain floor of Eq. 7, evaluated against the BM25-inferred task domain (§4.6), while mutation- and relevance-level concerns are delegated to the `write_safety` and `tool_relevance` rules (§3.5). The model therefore consumes no per-MCP vocabulary, no curated tool→capability map, and no ground-truth label—the property that makes the pipeline leak-free.

Identity infrastructure. PALADIN uses cryptographic agent identity (SPIFFE/SPIRE) as the source of ι in the request tuple (§3.1); identity is an *input* to the policy stack, not itself a contribution we evaluate. Each agent receives a SPIFFE ID encoding role and trust attributes (clearance, department, trust score); SPIRE issues SVIDs with automatic rotation and workload-selector attestation. We do not measure rotation overhead, attestation failure modes, or delegation-substrate properties; that work belongs to identity-infrastructure literature [30, 39], over which PEDIGREE-style multi-hop delegation could compose. Persona-to-domain allow-lists are released with the artefact.

Transport authentication (mTLS).. The same SPIRE-issued identities anchor the transport. Each workload presents its X.509 SVID to establish *mutual* TLS with the MCP invocation boundary, so both endpoints are cryptographically authenticated and every request travels an encrypted, identity-bound channel—the transport half of stage S_1 (Fig. 1). This is the transport realisation of zero-trust identity: the caller is trusted by neither network position nor a bearer token, but by a rotating certificate bound to its workload identity. Like the identity substrate itself, mTLS is trusted pass-through—we assume it and do not measure it; certificate issuance, rotation, and revocation are handled by SPIRE and lie outside the authorisation results. Folding it into S_1 keeps the model at six numbered stages (Fig. 1) while making explicit that authentication covers both *who* the agent is and the *channel* its invocation rides.

3.5 TRAC: a Task-Relational Admission Primitive

S6 (TRAC) evaluates typed predicates over $\Phi(r)$:

$$S_6(r) = \begin{cases} \text{DENY} & \text{if } \exists\, R_i \text{ s.t. triggered}(R_i) \wedge a_i = \text{DENY} \\ \text{ALLOW} & \text{otherwise.} \end{cases} \tag{6}$$

TRAC's capability model is deliberately *agnostic*: a capability is the pair $\{d\}$:$\{a\}$, where d is the MCP a tool belongs to and $a \in \{\text{read}, \text{write}\}$ is its action class, with the subsumption rule that a write also grants the read ($\{d\}$:write $\sqsupseteq$ $\{d\}$:read). There is no per-MCP vocabulary, no capability catalog, and no curated tool→capability map: the bundle's provided capabilities are $\text{HasCap} = \{\{d(t)\}:\{a(t)\} \mid t \in T\}$, derived mechanically from each tool's MCP and its read/write class (§3.4). Coverage is domain-only: the required capability for a task with inferred domain d_τ is the action floor $\{d_\tau\}$:read—any in-domain tool, read or write, satisfies it—and a hard violation is

$$\text{HardCapMissing}(r) \triangleq \{d_\tau\}\text{:read} \notin \text{HasCap}. \tag{7}$$

We deliberately do *not* infer read-vs-write *intent* from task text, nor gate on a weighted alignment score: those heuristics were substring-fragile and, for the domain signal, oracle-dependent. Mutation concerns are owned by the `write_safety` advisory and lexical task–bundle fit by `tool_relevance` (§3.5). d_τ itself is inferred from the task text alone (§4.6), so the entire coverage check is free of any ground-truth label. This is a property conventional RBAC

Table 3: PALADIN's four agnostic TRAC rules—declarative and extensible. The two DENY rules change the admission decision; the two *alert* rules only annotate the audit trail. Each conditions on the task↔bundle relation (`write_safety` on a within-bundle safety check)—the axis RBAC and ABAC do not provision as an attribute (§3.5).

Rule	Relation tested	Fires when	Effect
capability_coverage	task domain ↔ bundle domains	no tool in the task's domain	**DENY**
tool_relevance	task text ↔ tool descriptions	little text overlap	**DENY**
write_safety	destructive op ↔ verifying read	delete, no prior read	alert
action_coherence	task intent ↔ bundle actions	read task, mutating bundle	alert

(caller→tool) and ABAC do not *provision* as a standard, trusted attribute: whether the proposed bundle is coherent with the task's domain—expressible, as we show, over *derived* attributes on a standard engine (§5.4.3, §3.5).

TRAC (Task-Relational Access Control) conditions admission on the *measured relation between the task and the proposed tool bundle*—in-domain coverage, mutation safety, and task–tool relevance—which conventional RBAC (caller→tool) and ABAC do not *provision* as a standard attribute, and which is distinct from the TBAC/ReBAC/Purpose-BAC axes (§2, §3.5). We claim no new formal model: the contribution is empirical—identifying, enforcing, and measuring this relation as a load-bearing signal—enforceable on commodity engines. Its expressive content equals a typed Cedar [9] or OPA/Rego extension with custom builtins for continuous LLM-emitted scores; an OPA/Rego re-implementation of the identical rules-as-data reproduces every decision (§5.4.3), leaving only the substrate's *performance* cost to benchmark. The engine is *typed* (booleans, enums, reals), *staged* (facts derived in $\phi = S_5$ before any S_6 rule fires), *ordered* (enforcing rules short-circuit on the first denial), and *partitioned* into *enforcing* rules (change ALLOW/DENY) and *advisory* alert-only rules (Table 3); evaluation is $O(n)$ given pre-computed facts.

Scope. TRAC is a *capability-envelope* check—does the domain×action a bundle would exercise fall within the task's inferred domain?—not a prediction of whether a call is *safe* or a tool *benign*. It is thus *necessary, not sufficient*: an in-domain bundle can still misbehave, and clearing TRAC scopes an invocation without vouching for it. This is deliberate. Deciding whether an arbitrary, possibly-malicious payload is safe to execute from its metadata alone is not an access-control question and is intractable in general; TRAC instead gates the one static, decidable property that scoping *is*—capability↔task fit—and leaves outcome and integrity to the composed layers (output filtering, prompt-injection defence; §3.2). Being *preventive*—denying at the invocation boundary, before the call executes—a correct denial leaves nothing to undo. And because it reads the task text, it is only as trustworthy as that text's provenance: rather than assume that, we measure the boundary—an adversary who controls the task and pads it with the bundle's own descriptions drives TRAC's catch from 57.1% to 0.0% (§6.2), which is the composed prompt-injection layer's charge, not TRAC's.

These four rules instantiate TRAC concretely; they are declarative data, so an operator can retune thresholds or add rules without touching the engine. The two enforcing rules gate admission: `capability_coverage` denies a bundle when none of its tools operates in the task's domain—inferred from task text alone, so the check reads no ground-truth label (§4.6) and softened by *corroborated coverage* (§5.4.2)—while `tool_relevance` denies a bundle whose tools have low lexical (BM25) overlap with the task. The two advisory rules only annotate the audit trail: `write_safety` on a destructive operation lacking a verifying read, and `action_coherence` on a read-only-sounding task that selected mutating tools. All are fail-secure and deterministic—predicates default conservatively on missing facts, and identical facts yield identical, individually-explained decisions—and the set is illustrative rather than exhaustive: further task-relational predicates compose in the same typed, ordered form.

Engine portability. The feature set S6 exercises—typed continuous LLM-emitted scores, derived predicates as first-class rule inputs, ordered short-circuit denial with auditable triggered-rule trails, and a fixed pre-rule fact-derivation stage—is not exotic: each is native or a standard extension away in commodity policy engines (Cedar extension functions, OPA/Rego builtins and modules, Sentinel imports), none of which ships the full set out of the box. We therefore treat the engine as interchangeable and verify it directly—an OPA/Rego re-implementation of the identical rules-as-data reproduces every decision on a standard runtime (§5.4.3), so the substrate choice changes none of the empirical findings, and a Cedar port plus a performance benchmark are the remaining follow-ups.

The task-relational object, and how it is derived. TRAC's novelty is not a new engine but a new *object* of authorisation—the capability a bundle exercises *in service of* the task—derived without any privileged label. The two enforcing predicates are realised by Okapi BM25 [32] over public MCP metadata and the trusted task text: `capability_coverage` infers the task domain d_τ by scoring the task against each server's pooled tool names and descriptions and requires the bundle to cover it (Eq. 7, §4.6); `tool_relevance` scores each selected tool's description against the task and denies a bundle with negligible overlap. The action class $a(t)$ that types these facts comes from the same leak-free transformer—the Levin/VerbNet-grounded verb lexicon of §3.4. Because every input is either public (tool metadata) or trusted provenance (the task), no ground-truth field—domain, tool, or legitimacy label—ever reaches a decision: the property that makes the whole floor *leak-free*.

Why not simply add an ABAC attribute? ABAC authorises over attributes of the subject, object, action, and *environment*, and is deliberately extensible, so one might reasonably ask why the task–bundle relation is not merely another attribute—an *environment* attribute, since it is a property of neither the subject nor the object. We agree that is where it belongs; the distinction is not the *engine* (we enforce TRAC on an ABAC-class engine—OPA/Rego parity, §5.4.3) nor the attribute's *category*, but its *provenance*. ABAC gives coherence a *slot*—an environment condition the policy may reference—but not a *value*: every ABAC attribute is assumed to be supplied to the decision point by a trusted attribute authority (a PIP), and no off-the-shelf PIP emits "this bundle coheres with this free-text task." Nor can the value simply be declared—the task arrives as free text and a *semi-honest* agent controls any self-declared purpose (the Purpose-BAC [5] attribute), so a provisioned or declared coherence label cannot be trusted (§3.2). The signal is moreover *relational*

Table 4: Four steps of one mission (τ inferred to `grafana`), each a separate request through the stack. ✓ admit, × deny, – not evaluated (short-circuited); S_5 never denies and is omitted. Step A makes progress; each later step passes *every* earlier classical check and is denied by a progressively more specific layer. Verdicts are the released engine's.

	Proposed call(s)	S_1	S_2	S_3	S_4	S_6	Decision
A	`grafana: query_prometheus, get_dashboard_by_uid`	✓	✓	✓	✓	✓	**ALLOW**
B	`+stripe.create_refund`	✓	✓	×	–	–	**DENY** (RBAC)
C	`atlassian.jira_add_comment`	✓	✓	✓	×	–	**DENY** (ABAC)
D	`atlassian: jira_search, confluence_search`	✓	✓	✓	✓	×	**DENY** (TRAC)

and set-valued—a property of the whole proposed bundle *with respect to* the task, not of any single tool—so even *naming* it takes the task as an input and a computation over the entire bundle. TRAC's contribution is therefore the missing *PIP*: the *leak-free derivation* of this one environment attribute at invocation time—BM25 domain inference and VerbNet action-classing over public metadata and trusted task text, consuming no ground-truth label (§4.6)—together with the evidence that it is load-bearing. In ABAC terms we *do* add an attribute and enforce it as ordinary typed rules (§5.4.3); the work is not inventing the slot but *obtaining its value trustworthily*, which standard ABAC deployments assume is handed to them. It is equally distinct from the lineages closest to a "task-relational" claim—TBAC [41] (workflow position), ReBAC [15] (entity graph), Purpose-BAC [5] (*declared* purpose)—all of which condition on a provisioned or self-declared label, whereas TRAC gates on *measured* coherence (§2). The worked example makes this concrete: bundle D is authorisation-correct under *conventional* ABAC yet denied by TRAC alone.

3.6 A Request Through the Stack: A Worked Example

To make the model concrete we trace one agent through four steps of a single mission, each a separate invocation-time request (§3.1). The agent is the *Incident* persona (clearance L2, department Operations, trust 0.88, from the released policy), and the task is τ ="*Investigate why the checkout service is showing high request latency and a spike in 5xx error responses over the last hour.*" Leak-free BM25 (§4.6) infers d_τ = `grafana` from this text alone (score 11.1; next-best `azure` 6.1). Table 4 shows the per-stage outcome; every verdict is produced by the released engine on the released policy, not hand-derived.

A—progress. In-domain reads clear every stage: RBAC grants the Incident persona these Grafana tools, ABAC admits a medium-risk read at L2, and TRAC finds the bundle in-domain and relevant. The agent makes progress, as a program reads its own libraries uninterrupted.

B—classical denial (RBAC). Over-helpfully appending a cross-domain write, `stripe.create_refund`, is denied at S_3 on the classical triple—subject Incident, object `(stripe, create_refund)`, operation write—because the role holds no Stripe grant ("*Access to (stripe.create_refund) is not permitted for this identity*"). The step is denied; the mission continues through admitted calls.

C—context gate (ABAC). A benign write the role *does* grant, `jira_add_comment`, passes RBAC yet is denied at S_4: Atlassian is a high-risk resource and writing it requires L3 clearance, which the L2 agent lacks (rule `abac_clearance_write_high_risk`). This is a condition RBAC cannot express—defence in depth: the role grants the capability, but the attribute gate withholds it.

D—task-relational denial (TRAC). An Atlassian-only bundle the agent is fully *authorised* to run—RBAC and *conventional* ABAC both admit these reads—is nonetheless denied at S_6: the task is in the `grafana` domain, so the Atlassian bundle covers no required capability (`grafana:read` is absent from HasCap; `HardCapMissing`, Eq. 7) and the `capability_coverage` rule fires. This is the paper's thesis in one row: the bundle is authorisation-correct yet task-incoherent, a distinction only the task-relational layer draws.

The escalation A→D is the point: each denied request passed every *earlier* check and was caught only by a later, more specific layer; and the more privileged the agent, the more the classical layers admit and the more the task-relational layer carries (§5.3.3).

4 Experimental Methodology

This section specifies the end-to-end simulation setup: the corpus and personas (§4.1), the concrete policies enforced (§4.2), the dual-mode judging/selection protocol (§4.3), the ablation design (§4.4), the single confusion-matrix convention—security as precision, availability as recall (§4.5)—and the leak-free domain inference that keeps the whole floor label-free (§4.6).

4.1 Dataset, Personas, and Model Panel

We evaluate on ASTRA v0.3 [11], a public corpus of 1,157 agentic task-tool matching tasks across 8 MCP server domains (194 tools: grafana, atlassian, azure, mongodb, stripe, notion, hummingbot, wikipedia). ASTRA is, to our knowledge, the only public corpus pairing ground-truth tool selections with labelled `wrong` (same-domain) and `null` (cross-domain) negatives across multiple domains—the exact structure an authorisation evaluation needs, and one that injection benchmarks [10, 34, 49] lack. Onto this we layer six personas (DevOps L3, Incident L2, Finance L2, Research L1, Automation Gateway L3, Security Engine L3)—with released SPIFFE IDs, clearance, and trust scores—to turn tool selection into an *authorisation* benchmark; every persona–domain policy is published for audit.

Corpus composition. Every ASTRA task pairs a natural-language request with a candidate tool selection carrying a ground-truth `match_tag`. The 1,157 tasks partition into 579 `correct` selections (50%), 463 `wrong` selections (40%; same-MCP wrong tools—hard within-domain negatives), and 115 `null` selections (10%; unrelated-MCP tools—gross cross-domain negatives); among the negatives this is the 80:20 wrong-to-null split fixed by ASTRA's construction.

The model panel spans six models across three vendors: three OpenAI generations—**gpt-3.5-turbo-16k** (2023), **gpt-4o** (2024-05; ASTRA's reference model, enabling direct replication, §5.2.2), and **gpt-5.4** (2026)—plus a cross-vendor set of **claude-opus-4.8**, **claude-sonnet-4.6** (Anthropic), and **gemini-2.5-pro** (Google), the 2026 models served through Microsoft Azure AI Foundry. All inference uses temperature 0.0, JSON-schema-constrained output, and a single retry on parse failure; no system prompt beyond the documented PALADIN protocol is injected. Two models anchor the whole-flow analysis (§5.3): **claude-opus-4.8**, the highest-availability validator alone (recall 0.971, §5.2), and

Table 5: The six released personas (`policies/rbac.yaml`, `experiment_config.py`): SPIFFE identity, clearance, department, trust, and the least-privilege MCP-domain grant. *R* = read, *W* = write; every persona terminates in `default_deny`.

Persona	Clr	Dept	Trust	Domain grant
DevOps	L3	Engineering	1.00	grafana, atlassian, azure, mongodb (R/W)
Incident	L2	Operations	0.88	grafana, atlassian (R + benign W)
Finance	L2	Finance	0.95	stripe, hummingbot (R + non-destructive W)
Research	L1	Research	0.75	wikipedia, notion (R)
Gateway	L3	Infrastructure	1.00	all 8 (R); azure, mongodb (W)
Security	L3	Security	1.00	all 8 (R, audit)

gpt-4o, the ASTRA reference. Two panel caveats carry into the results: gpt-3.5-turbo-16k has no selection run—its ∼ 21k-token candidate-catalog prompt exceeds the 16,385-token context (§5.2)—and gemini-2.5-pro's selection was produced by an in-environment batched proxy. Open-weights models (Llama, Mistral) remain deferred (§6.4).

4.2 Policy Details

The deployment enforces a single released policy set (`policies/*.yaml`); we summarise it here because the ablations (§4.4) and the load-bearing analysis (§5.3.3) turn on its structure.

Identity and RBAC. Each of the six personas (Table 5) carries a SPIFFE ID, a clearance level, a department, and a trust score, and receives a least-privilege slice of the eight MCP domains that always terminates in `default_deny`. Grants encode role *and* destructiveness: the Incident agent may comment on and transition Jira issues but not create, update, or delete them; Finance may manage invoices but not refund, cancel, or execute trades; Research, the Gateway's non-infrastructure domains, and the Security engine are read-only.

ABAC. Six attribute rules, evaluated as rules-as-data, deny across three families: *department isolation* (financial/PCI-DSS resources stay in Finance; infrastructure writes stay in Engineering or Infrastructure), *clearance gates* (high-risk reads require ≥L2, high-risk writes require L3, destructive operations require L3), and a *trust gate* (any write requires trust ≥ 0.8). Resource risk follows `mcp_risk_levels.yaml`: atlassian, azure, mongodb, stripe, and hummingbot are high-risk, grafana and notion medium, wikipedia low.

TRAC. Four task-relational rules (§3.5) close the stack: two *enforcing*—`capability_coverage` (deny a bundle that cannot operate in the task's BM25-inferred domain) and `tool_relevance` (deny a bundle whose tools have negligible BM25 overlap with the task)—and two *advisory*—`write_safety` and `action_coherence`—that annotate the audit trail without blocking; a corroborated-coverage rescue (mean tool relevance ≥ 4.0, §5.4.2) reverses a domain-mismatch denial. All thresholds are fixed *ex ante* and retunable without touching the engine.

4.3 Dual-Mode Evaluation

The six personas expand the 1,157 tasks to 6,942 (task, persona) rows. Each row is evaluated under one of two modes; row identifiers are shared across modes so that paired comparisons are well-defined.

Table 6: The admission confusion matrix. Positive = *valid* (task-correct *and* persona-authorised); predict-positive = *allowed*. The two error cells have opposite costs: FP (security failure) is a not-valid request the stack admits; FN (false block) is legitimate work the stack denies.

	allowed	**denied**
valid (correct ∧ authorised)	TP correct admit	FN false block
not-valid (wrong/null ∨ unauth.)	FP security failure	TN correct block

Validation mode. The LLM is given the task text *together with* a candidate tool bundle and emits a JSON verdict (`validation_failed` plus issue codes). The LLM judges, not selects. Ground truth is the bundle's `match_tag` (§4.1): `correct` bundles should validate, while `wrong` (same-MCP) and `null` (cross-MCP) bundles should be rejected—so each of the 6,942 rows is a labelled admit/deny decision.

Selection mode. The LLM is given the task text *only* and must *generate* a tool bundle as a JSON list, routed through the *same* deterministic pipeline with no verdict gate—in selection the model *is* the requester. Selection is strictly harder: exact tool recovery is low even for the strongest model (best-in-panel Jaccard 0.24, §5.2), because the model actively emits same-domain wrong-tool bundles the agnostic stack cannot separate. The generated bundle is scored on tool recovery *before* gating (§4.5), and its gated survivors are then read as a funnel (§5.3.2). We evaluate selection on the 6,942 rows.

4.4 Ablations

Layer configurations. Each row records every layer's state, so from a single evaluation we recover offline both the cumulative survivor funnel (RBAC → ABAC → LLM verdict → TRAC) and every subset of the deterministic stack, hence each layer's order-independent drop-one marginal (§5.3.3). The LLM-alone configuration—the verdict with no policy layers—is meaningful only in validation; in selection the model is the requester, so no standalone admit/deny verdict exists. Corpus: the six-model panel over 6,942 rows per (model, mode), ≈ 114,000 policy decisions in all.

4.5 Metrics

We score two objects in turn—the *model* as a standalone component (§5.2) and the *system* as an admission control (§5.3)—under a single confusion-matrix convention fixed here.

The admission confusion matrix. Every (task, persona) request carries a ground-truth label and receives a pipeline decision (Table 6). The label is **valid** iff the candidate bundle is task-correct (`match_tag=correct`) *and* the persona is authorised for the task's domain (the released persona–domain allow-list, §4.2); otherwise it is **not-valid**—a `wrong`/`null` bundle, or a correct bundle the role may not run. The decision is **allowed** or **denied**. Taking *valid* as the positive class and *allowed* as predicting positive gives four cells: a valid request allowed is a true positive (TP); valid but denied is a false negative (FN, a *false block*); not-valid but allowed is a false positive (FP, a *security failure*); not-valid and denied is a true negative (TN).

Security and availability. We read the standard metrics through the two error cells' opposite costs:

$$\text{Precision (security)} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall (availability)} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{8}$$

$$F_1 = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}, \qquad \text{SecFail} = \frac{\text{FP}}{\text{FP} + \text{TN}}. \tag{9}$$

Precision is security: of everything admitted, the share that was truly valid—an FP is a not-valid request let through. **Recall is availability**: of everything truly valid, the share admitted—an FN denies legitimate work. The *security-failure rate* SecFail = FP/(FP + TN) is the fraction of not-valid requests admitted; it is the single number the deterministic floor bounds (§5.1) and the quantity a permissive LLM inflates. Accuracy is (TP + TN)/N. Raising security (precision, lower SecFail) by denying more trades against availability (recall); the operating point is set by policy, not architecture (§6.1).

Model-alone scoring (§5.2). With no policy stack the LLM is scored at *task* resolution over the 1,157-task corpus (persona-invariant, §5.2.1). In *validation* the positive class is the correct bundle and we report Precision, Recall, F_1, Accuracy of the model's admit/deny verdict; this task-resolution view coincides with ASTRA's permissivity setting, enabling direct replication (§5.2.2). In *selection* no admit/deny verdict exists, so the generated bundle is scored on tool recovery: the graded count of the task's three gold tools returned (3/3, 2/3, 1/3, 0/3), summarised by exact-match rate and Jaccard.

System scoring (§5.1, 5.3). As an admission control the pipeline is read as a *survivor funnel*: per persona, how many requests are still admitted after each cumulative gate (RBAC → ABAC → LLM verdict → TRAC). On the not-valid subset the final survivors are the security failures that set SecFail and precision; on the valid subset the requests denied along the way are the availability cost that sets recall. The funnel is the raw-count form of the confusion matrix and makes each layer's marginal contribution directly visible (§5.3.3).

Label independence. The positive class combines ASTRA's `match_tag` with the persona–domain allow-list that also sources the RBAC grants, so some RBAC denials coincide with the label by construction; §5.3.3 shows the layer conclusions survive an independent `match_tag`-only label and a label-free RBAC re-authoring.

Reproducibility. All thresholds were fixed *ex ante* (config commit hash recorded with the artefact, §6.4). Raw per-row logs, the BM25 retriever, the splits file, and analysis scripts are released.

4.6 Leak-Free Domain Inference

The agnostic capability check (§3.5) requires the task's domain d_τ. We infer it *from the task text alone,* consuming no ground-truth field: each MCP server's tool names and descriptions form one BM25 document, the task text is the query, and d_τ is the top-scoring domain among the deployment's RBAC-entitled MCPs—or `uncertain` when the top two scores fall within 15%, in which case coverage abstains rather than deny on a coin-flip. This closes the only oracle dependency in the pipeline. An earlier configuration substituted the gold MCP (`gt_mcps[0]`); the present task-text inference removes it entirely, so no gold field—`match_tag`, `is_legitimate`, tool, or domain—reaches a decision path. Inference is intentionally imperfect (~60–70% top-1 on ASTRA), which is the honest cost of forgoing the oracle; when the inferred domain is wrong but the selected tools are strongly task-relevant, corroborated coverage (§5.4.2) recovers the denial.

5 Results and Analysis

We report results in four steps, each presenting its table *before* interpreting it: the deterministic floor alone (§5.1), the LLM alone (§5.2), the two composed into the whole flow (§5.3), and an in-depth robustness analysis (§5.4). All numbers are computed from the released raw logs under the single confusion-matrix convention of §4.5—security = precision, availability = recall, and the security-failure rate (Eq. 9) the fraction of not-valid requests admitted. Each system table is a per-persona *survivor funnel*: the count still admitted after each cumulative gate. Full per-row confusion matrices, layer marginals, and bootstrap CIs ship with the artefact.

5.1 The Deterministic Floor

We first characterise the pipeline's deterministic layers *alone*—RBAC ∧ ABAC ∧ TRAC with no model in the loop. Because these layers condition only on the persona, the candidate bundle, and the leak-free inferred task domain (§4.6), they return bit-identical decisions for every model: one *model-independent* floor over all 6,942 rows, verified identical across all six panel logs. Aggregated, the floor gives precision (security) 0.63, recall (availability) 0.45, and security-failure rate 10.3%. This is the floor the rest of the section builds on: how each model sits relative to it unaided (§5.2), and how composing the two sharpens it (§5.3). The operating-point map situates the floor; Table 7 then decomposes it into per-persona funnels.

5.1.1 Operating-Point Map: Security Failure vs. Availability. Figure 2 plots each configuration's security-failure rate against its availability (recall—the fraction of valid, task-correct *and* authorised, requests admitted). The three deterministic points are model-independent; the per-model LLM-only and composed points vary by model.

The deterministic layers trace a *monotone* frontier—each added layer lowers the security-failure rate (54% → 31% → 10%) at an availability cost; every LLM-only point sits well above it whatever its availability, while composing the model behind the floor (FULL) collapses all six models across three vendors into a tight low-security-failure cluster ($\leq$ 7.3%), none exceeding the 10.3% floor (§5.3.3).

The security/availability trade-off. The floor's availability—recall 0.448, i.e. 44.8% of valid (task-correct *and* authorised) requests admitted—is deliberately conservative. Most of the shortfall is least-privilege enforcement: RBAC/ABAC deny correct bundles a role is not entitled to run, which is enforcement working as intended rather than lost availability. The residual denial of *authorised* correct work comes from the leak-free task-relational layer's imperfect domain inference and is tunable (§6.1); the low admission is a deliberate max-security operating point on a policy-navigable frontier, not a fixed limit.

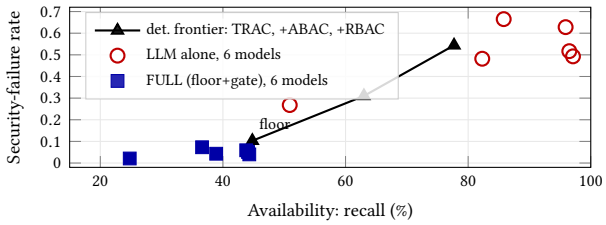

Figure 2: Operating-point map: security-failure rate vs. availability (recall, % of valid requests admitted). The model-independent deterministic frontier (TRAC → +ABAC → +RBAC) is monotone; every LLM-only validator sits well above it at high security-failure *regardless* of availability, while composing each model's verdict behind the floor (FULL) pulls all six models, across three vendors, into a tight low-security-failure cluster (≤ 7.3%), none exceeding the 10.3% floor. Availability counts *every* denied valid request, including least-privilege denials by RBAC/ABAC. Values: Table 9; replay via `scripts/sweep_op_points.py`.

5.1.2 Per-Persona Floor. The aggregate floor is a mean over six personas whose funnels differ sharply (Table 7); because the deterministic layers condition only on persona, bundle, and inferred domain, this decomposition is model-independent. Narrowly-scoped roles admit very little: Research (L1) retains only 22/1157 ground-truth bundles after the selection floor and 11 after validation—RBAC's tight grant leaves few same-domain wrong-tool bundles for the semantic layer to miss. Broad L3 infrastructure roles (DevOps, Gateway, Security) admit far more at every gate—DevOps 632, Gateway 487, Security 367 selection survivors—shifting the discrimination burden onto the leak-free TRAC predicate. This per-role spread under one fixed policy is why an agnostic floor is a *floor*, not a solution: the model gate must equalise it (§5.3).

5.2 The LLM Alone

Removing the deterministic floor and letting the model judge or select unaided shows how a single LLM behaves as a control. We score it at *task* resolution over the 1,157-task corpus (persona-invariant, §5.2.1) with the correct bundle as the positive class—its intrinsic semantic-judgment quality, independent of any policy.

Table 8 reports both modes for all six models: *validation*, where the model judges a candidate bundle and has a scoreable admit/deny verdict (precision, recall, F_1, accuracy); and *selection*, where it generates the bundle and is scored on recovery of the task's three gold tools (graded $k/3$ counts, exact-match rate, Jaccard). gpt-3.5-turbo-16k has no selection run—its candidate-catalog prompt exceeds context.

5.2.1 Task Resolution vs. Authorisation Resolution. The verdict is *persona-invariant*: for every one of the 1,157 tasks the model returns an *identical* admit/deny for the candidate bundle whichever of the six personas issued it (1157/1157 tasks, every panel model). The 6,942-row system view is therefore 1,157 semantic judgments replicated six-fold—the model decides at *task* resolution, blind to the requester. Authorisation, however, is persona-relative: on each of the 579 task-*correct* bundles legitimacy depends on the role—valid for some personas, a least-privilege *violation* for others (§4.1)—yet the persona-blind model admits the correct bundle for all six, passing the violations silently. Scored as an *authorisation* control

Table 7: The deterministic floor as per-persona survivor funnels (model-independent, n = 6,942). Each cell is the number of requests *still admitted* after that cumulative gate. *Selection*: every task's ground-truth bundle (perfect selection), gated RBAC→ABAC→TRAC. *Validation*: all candidate bundles with an oracle rejecting every incorrect one, gated RBAC→ABAC→LLM→TRAC. Narrowly-scoped roles (Research, Finance) admit few; broad L3 roles (DevOps, Gateway, Security) admit many, shifting the burden onto TRAC. Totals give floor precision (security) 0.63, recall (availability) 0.45, security-failure 10.3%.

Persona	total	RBAC	ABAC	LLM	TRAC
Selection — ground-truth bundles; gates RBAC→ABAC→TRAC					
DevOps *(L3)*	1157	770	770	—	632
Incident *(L2)*	1157	307	260	—	231
Finance *(L2)*	1157	141	43	—	41
Research *(L1)*	1157	63	63	—	22
Gateway *(L3)*	1157	628	616	—	487
Security *(L3)*	1157	510	454	—	367
Total	6942	2419	2206	—	1780
Validation — all candidates + oracle reject-all; gates RBAC→ABAC→LLM→TRAC					
DevOps *(L3)*	1157	748	748	384	314
Incident *(L2)*	1157	303	254	129	114
Finance *(L2)*	1157	124	37	21	20
Research *(L1)*	1157	78	78	30	11
Gateway *(L3)*	1157	639	627	306	242
Security *(L3)*	1157	514	462	228	185
Total	6942	2406	2206	1098	886

Table 8: The LLM alone, both modes, at task resolution (n = 1,157 tasks, persona-invariant; positive class = correct bundle). *Validation* (top): the model judges a candidate bundle—precision, recall, F_1, accuracy of its admit/deny verdict. *Selection* (bottom): the model generates the bundle and is scored on recovery of the task's three gold tools ($k/3$ counts over all 1,157 tasks, exact-match rate, mean Jaccard). Recall (availability) is high for the 2026 cross-vendor models but precision (security) never exceeds 0.78; selection is far weaker (best exact-match 9.4%). Neither mode is monotone in recency (§5.2.3). Positive = valid (persona-relative) system scoring of these same runs is §5.2.1.

Validation — LLM as judge	P	R	F_1	Acc
gpt-3.5-turbo-16k *(OpenAI '23)*	0.705	0.815	0.756	0.736
gpt-4o *(OpenAI '24)*	0.740	0.497	0.595	0.661
gpt-5.4 *(OpenAI '26)*	0.583	0.839	0.688	0.620
claude-opus-4.8 *(Anthropic '26)*	0.772	**0.971**	**0.860**	**0.842**
claude-sonnet-4.6 *(Anthropic '26)*	0.748	0.964	0.842	0.819
gemini-2.5-pro *(Google '26)*	0.666	0.962	0.787	0.740

Selection — LLM as generator	3/3	2/3	1/3	0/3	Exact	Jacc.
gpt-3.5-turbo-16k *('23)*	*n/a — catalog prompt > context*					
gpt-4o *('24)*	52	189	304	612	4.5%	0.179
gpt-5.4 *('26)*	66	206	292	593	5.7%	0.197
claude-opus-4.8 *('26)*	109	228	260	560	**9.4%**	**0.238**
claude-sonnet-4.6 *('26)*	93	221	290	553	8.0%	0.226
gemini-2.5-pro *('26)*	28	104	274	751	2.4%	0.117

under the system convention (positive = valid, §4.5), the unaided model's security-failure rate spans 26.8–66.5% across the panel (gpt-4o best at 26.8%, gpt-5.4 worst at 66.5%)—the mechanistic form of our thesis (§1.1): *semantic correctness is not authorisation correctness.* RBAC exists to supply exactly the persona resolution the model structurally lacks (§5.3.3).

5.2.2 Independent Replication of ASTRA's LLM-ResM Benchmark. At task resolution (positive = correct bundle, $n = 1{,}157$, matching ASTRA's permissivity setting), our gpt-4o reaches $F_1 = 0.595$ vs. ASTRA's published 0.67—a 0.075 gap under a strictly harder protocol (ASTRA AND-aggregates three per-tool verdicts, inflating precision; we use one per-bundle call), so our number is the expected lower bound. The precision–recall signature replicates: high precision (0.740 vs. 0.81), low recall (0.497 vs. 0.57), under-scoping dominant; the two other OpenAI generations bracket gpt-4o (0.756, 0.688). A reproduction of ASTRA's under-scoping finding, not a refutation.

5.2.3 Non-Monotone LLM-Only Behaviour with Model Recency. A clean side-effect of the three OpenAI generations: LLM-only safety is *non-monotone* in recency (the ordering differs by metric). By semantic F_1 (positive = correct), gpt-3.5-turbo-16k (2023, 0.756) beats gpt-4o (2024, 0.595) and gpt-5.4 (2026, 0.688); as an authorisation control (security-failure rate), gpt-4o is strongest (26.8%) yet the 2026 gpt-5.4 is the *most* permissive (66.5% vs. 26.8%). We cannot causally attribute this without a controlled fine-tune, but the lesson holds across vendors—every 2026 model is a *more* permissive validator than the 2023 one (§5.3.1)—so *a single-LLM validator cannot be assumed to improve as base models advance.*

5.3 The Whole Flow: Composition and Recovery

Composing the two—the model's verdict *behind* the deterministic floor—is the deployed system. We read the composition three ways, each results-first: a cross-vendor validation panel showing the floor bounds every model's security-failure rate whatever its solo behaviour (§5.3.1); per-model survivor funnels for the two anchor models—claude-opus-4.8 (highest availability) and gpt-4o (the ASTRA reference)—that show *where* each layer catches (§5.3.2); and a model-independent layer ablation isolating each layer's marginal contribution (§5.3.3).

5.3.1 Cross-Vendor Validation: The Floor Bounds Security Failure. PALADIN is deployed as FULL = the model's own verdict *behind* the deterministic floor; it complements the LLM rather than replacing it. Table 9 reads that composition across six models spanning three vendors, each judging every candidate bundle *blind* to the ground-truth label with the *same* `ValidationService` rubric (verdicts frozen before scoring and released).

Alone, each model is an unreliable control: security-failure spans 26.8% (gpt-4o) to 66.5% (gpt-5.4), non-OpenAI models interleaved throughout, no ordering by vendor or recency. Behind the floor, the same verdict is bounded to security-failure ≤ 7.3% for all six, none exceeding the 10.3% floor—because FULL = floor ∧ verdict is a conjunction that can only *add* denials, so a capable judge (gpt-4o) sharpens security-failure to 2.1% while even a permissive one cannot lift FULL above the floor (P1, §3.1). The composition is *leak-free*: the domain driving coverage is inferred from task text by BM25 over the public MCP catalog (§4.6), so the floor sits at 10.3% rather than zero—the irreducible semantic floor the verdict, not the policy, must close.

Table 9: Cross-vendor validation composition (system convention, positive = valid, $n = 6{,}942$ per model). Six models, three vendors, each judging blind with the *same* rubric. LLM alone = the verdict with no policy; FULL = the same verdict behind the model-independent floor (listed once). P_{sec} is precision (security), R_{av} recall (availability), sec-fail the fraction of not-valid admitted. Every model is permissive alone (sec-fail 0.27–0.67) yet the floor bounds all six to ≤ 7.3% once composed; none exceeds the 10.3% floor. Composition trades availability for security by model: the strictest judge (gpt-4o) reaches sec-fail 0.021 at R_{av} 0.248, while claude-opus-4.8 holds R_{av} 0.443 at sec-fail 0.040. Task-level bootstrap CIs ship with the artefact.

	alone	FULL (floor+gate)		
Model (vendor, year)	**sec-fail**	**P_{sec}**	**R_{av}**	**sec-fail**
Deterministic floor *(model-independent)*: P_{sec} 0.634, R_{av} 0.448, sec-fail 0.103				
gpt-3.5-turbo-16k *(OpenAI '23)*	0.482	0.780	0.389	0.043
gpt-4o *(OpenAI '24)*	0.268	0.824	0.248	**0.021**
gpt-5.4 *(OpenAI '26)*	0.665	0.665	0.366	0.073
claude-opus-4.8 *(Anthropic '26)*	0.493	0.816	**0.443**	0.040
claude-sonnet-4.6 *(Anthropic '26)*	0.517	0.775	0.441	0.051
gemini-2.5-pro *(Google '26)*	0.628	0.747	0.438	0.059

Security vs. availability is now a per-model choice. Composition sharpens security for every model, but at a recall (availability) cost that differs sharply: gpt-4o drives security-failure to 2.1% yet admits only 24.8% of valid work, whereas claude-opus-4.8 holds availability at 44.3% for a near-identical 4.0% security-failure. opus is thus the highest-availability composition on the panel and gpt-4o the strictest—the two we trace as funnels next (§5.3.2).

Operating point. The floor profile comes from the released policy—the `capability_coverage` domain floor, the `tool_relevance` BM25 cutoff (< 1.0), and the corroborated-coverage rescue bar (≥ 4.0), all editable in `policies/trac_rules.yaml` and fixed *ex ante*; §5.4.2 gives the Pareto refinement that sets it.

5.3.2 Whole-Flow Funnels for the Anchor Models. Tables 10 and 11 trace the two anchor models through the whole flow as survivor funnels: the count still admitted after each cumulative gate, restricted to the requests that stress the system—the 578 incorrect (wrong/null) tasks per persona in validation, and each model's own wrong/null bundles in selection. The final column is the residual security failures.

Because RBAC and ABAC condition only on persona and bundle, their columns are *identical* across the two models (1200, 1108); the models differ only at the LLM verdict and the TRAC catch. gpt-4o's stricter verdict admits far fewer incorrect tasks at the LLM gate (1108 → 216 vs. opus 1108 → 372) and leaves a lower residual after TRAC (105 vs. 198)—the security half of the trade in Table 9, bought with the availability opus retains. ABAC's marginal is visibly small (1200 → 1108, 92 caught) because RBAC already denies most of

Table 10: claude-opus-4.8 whole-flow survivor funnels (the highest-availability model). Each cell is the number *still admitted* after that cumulative gate. *Validation*: the 578 incorrect tasks per persona (n = 3,468), gated RBAC→ABAC→LLM verdict→TRAC; 198 survive. *Selection*: opus's own wrong/null bundles, gated RBAC→ABAC→TRAC; 995 survive. RBAC and ABAC condition only on persona and bundle, so their columns are model-independent; ABAC removes little beyond RBAC (1108 vs. 1200), the backstop pattern of §5.3.3.

Persona	total	RBAC	ABAC	LLM	TRAC
Validation — incorrect tasks; gates RBAC→ABAC→LLM→TRAC					
DevOps *(L3)*	578	364	364	98	55
Incident *(L2)*	578	150	125	45	26
Finance *(L2)*	578	52	16	3	3
Research *(L1)*	578	48	48	23	7
Gateway *(L3)*	578	327	321	109	56
Security *(L3)*	578	259	234	94	51
Total	3468	1200	1108	372	198
Selection — opus's wrong/null bundles; gates RBAC→ABAC→TRAC					
DevOps *(L3)*	578	383	383	—	316
Incident *(L2)*	578	179	155	—	132
Finance *(L2)*	578	81	49	—	43
Research *(L1)*	578	32	32	—	11
Gateway *(L3)*	578	367	342	—	266
Security *(L3)*	578	320	282	—	227
Total	3468	1362	1243	—	995

Table 11: gpt-4o whole-flow survivor funnels (the ASTRA reference, strictest composition). Columns and cohorts as in Table 10. The RBAC and ABAC columns are *identical* to opus's (1200, 1108)—those layers never see the model—so the two models diverge only at the LLM verdict and TRAC: gpt-4o's stricter verdict admits far fewer incorrect tasks (1108 → 216 vs. opus 1108 → 372) and leaves a lower residual (105 vs. 198).

Persona	total	RBAC	ABAC	LLM	TRAC
Validation — incorrect tasks; gates RBAC→ABAC→LLM→TRAC					
DevOps *(L3)*	578	364	364	66	34
Incident *(L2)*	578	150	125	33	16
Finance *(L2)*	578	52	16	2	1
Research *(L1)*	578	48	48	7	2
Gateway *(L3)*	578	327	321	63	31
Security *(L3)*	578	259	234	45	21
Total	3468	1200	1108	216	105
Selection — gpt-4o's wrong/null bundles; gates RBAC→ABAC→TRAC					
DevOps *(L3)*	578	391	391	—	313
Incident *(L2)*	578	181	165	—	141
Finance *(L2)*	578	71	41	—	37
Research *(L1)*	578	40	40	—	15
Gateway *(L3)*	578	370	350	—	268
Security *(L3)*	578	336	309	—	240
Total	3468	1389	1296	—	1014

what ABAC would—the backstop pattern quantified next (§5.3.3). TRAC then catches a further 174 (opus) and 111 (gpt-4o) incorrect tasks the verdict admitted, the task-relational layer working behind the model.

Selection. When the model *generates* the bundle there is no verdict gate, yet the same RBAC→ABAC→TRAC floor still removes 2,473/3,468 of opus's wrong bundles and 2,454/3,468 of gpt-4o's,

Table 12: Full-factorial layer ablation on the deterministic stack (validation, n = 6,942; model-independent). R_{av} is recall (availability); sec-fail is the security-failure rate (lower is safer; numerically identical to the earlier security-failure metric). *Every* ablation is worse on security than the full stack: conventional access control alone (RBAC+ABAC, no task-relational layer) leaves sec-fail at 0.223—more than double the full stack's 0.103, the operating point of any conventional AC method without the predicate (§2.1). The drop-one column gives the omitted layer's order-independent marginal (sec-fail of the full stack minus this row). Frontier in Fig. 3; per-model composition in Table 9.

Configuration	R_{av}	sec-fail	drop-one
No policy (all open)	1.000	1.000	—
RBAC only	0.610	0.242	—
TRAC only	0.777	0.544	—
ABAC only	0.810	0.610	—
RBAC+TRAC *(−ABAC)*	0.496	0.112	ABAC −0.9pp
RBAC+ABAC *(−TRAC)*	0.555	0.223	TRAC −12.0pp
ABAC+TRAC *(−RBAC)*	0.630	0.310	RBAC −20.7pp
RBAC ∧ ABAC ∧ TRAC (full)	0.448	**0.103**	—

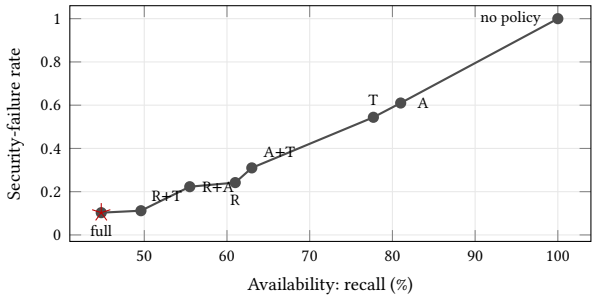

Figure 3: Ablation operating points: security-failure rate vs. availability (recall) for all eight subsets of the deterministic stack (validation; model-independent; R = RBAC, A = ABAC, T = TRAC). Adding a layer lowers the security-failure rate at an availability cost, tracing a monotone frontier with the full stack (star) at the secure end (sec-fail 0.103). Most of the availability gap is least-privilege denial of correct-but-*unauthorised* bundles—enforcement working as intended (§5.1.1). Values: Table 12.

so the deterministic stack recovers generation failures as well as verdict failures without re-tuning. (An earlier retrieval-augmented in-context-learning result—BM25 task–tool exemplars lifting gpt-5.4 selection exact-match 10.3 → 39.1% so the floor sharpens to security-failure 6.6%—is recomputable from the released logs and parked pending the funnel presentation.)

5.3.3 Which Layers Are Load-Bearing. Table 12 scores all eight subsets of the deterministic stack (validation; deterministic, hence model-independent): *every* ablation is strictly worse on security than the full stack, and conventional access control alone—RBAC+ABAC with no task-relational layer—leaves security-failure at 0.223, more than double the full stack's 0.103 (frontier

in Fig. 3). The honest per-layer measure is the order-independent *drop-one* marginal—the security-failure of the full stack vs. the stack with exactly that one layer removed. RBAC (−20.7pp, 95% CI [−22.6, −19.0]) and **TRAC (−12.0pp,** [−13.5, −10.6]**)** are each individually load-bearing—both CIs exclude 0—whereas **ABAC's drop-one marginal is only** −0.9**pp** ([−1.3, −0.6]), because ABAC and RBAC *overlap*: most attribute violations ABAC would catch are already caught by RBAC. ABAC is therefore a *backstop*: near-redundant given RBAC on this benchmark, but load-bearing when RBAC is absent (removing ABAC once RBAC is already gone costs a further 23.4pp). This same overlap surfaces concretely in the funnels of §5.3.2, where ABAC removes only 92 of 1,200 RBAC survivors.

Two readings follow. TRAC's isolated −12.0pp is direct evidence the task-relational axis catches violations conventional RBAC+ABAC cannot—measured, not asserted. And the gap between ABAC's first-firing share and its drop-one marginal instances the attribution pitfall of §5.4.1: a layer can look essential under one lens and near-redundant under another, so both must be reported—guidance not "never drop a layer" but "RBAC and TRAC are individually load-bearing; ABAC hedges RBAC." This also corrects the earlier oracle-aided policy, in which an alignment score did most of the validation work and RBAC/ABAC were near-redundant; removing the oracle redistributes the security load onto identity and the task-relational predicate—exactly where a zero-trust deployment can audit it.

Is the load-bearing pattern a label artefact? Two decouplings say no. Re-authoring RBAC *independently* of the label (coarse job-function roles that *over-provision* relative to the fine-grained allow-list) shrinks RBAC's drop-one marginal (−20.7 → −11.3pp), but the slack redistributes onto TRAC (−12.0 → −21.5pp) and ABAC (−0.9 → −9.1pp) and the floor degrades gracefully (security-failure 10.3 → 19.7%) while admitting strictly more legitimate work—no single layer's exact tuning is load-bearing. Decoupling the *label* itself from all policy (legitimacy from ASTRA's `match_tag` alone, dropping the persona–domain criterion) makes **TRAC's marginal *rise* to** −17.2**pp** [−19.1, −15.5]**, the largest of the three** (paired task-level bootstrap 95% CI clear of 0; RBAC falls to −14.6pp [−16.5, −12.7]): under a label no policy artefact authored, the task-relational predicate is the *most* load-bearing layer—the opposite of a circularity artefact.

The enforcing surface is not only lexical. Two TRAC rules reason about *action semantics*: `action_coherence` (read-intent task, mutating tools) and `write_safety` (destructive op without a preceding read). Making `action_coherence` enforcing catches 39 same-domain violations the BM25 predicates and RBAC+ABAC all miss, at precision 0.62 (1.69× the approved base rate), lowering security-failure 10.3 → 9.5% at 24 new legitimate blocks—structural coherence beyond BM25. (`write_safety` is weaker, precision 0.31, so advisory.)

5.4 In-Depth Analysis

Four analyses probe the result's robustness: why first-firing attribution misranks the layers, how corroborated coverage recovers mis-routed legitimate bundles, why the guarantee is a property of the *policy* rather than our engine (verified OPA/Rego parity), and the dataset ceiling that bounds selection exact-match.

5.4.1 The Attribution Paradox. Marginal necessity and first-firing attribution disagree sharply. Attributing every floor denial to the first layer that fires under short-circuit order (RBAC→ABAC→TRAC), RBAC claims 82% of denials, TRAC 15%, and ABAC only 4%. Yet the marginal picture (§5.3.3) is the opposite: ABAC, which first-fires on a mere 4% of denials, carries the largest *successive* marginal (−23.4pp—removing ABAC once RBAC is already absent), even though its order-independent *drop-one* marginal is only −0.9pp. The reason is pipeline order—most rows ABAC would catch are intercepted earlier by RBAC, so ABAC's unique value is invisible to a denial-source chart and surfaces only when RBAC is removed. First-firing attribution, common in the agent-safety literature, therefore systematically misranks layers; both attribution and the marginal security-failure change must be reported.

5.4.2 Corroborated Coverage. The leak-free domain classifier (§4.6) is intentionally imperfect: a business-phrased task can route to a lexically-similar but wrong MCP, so `capability_coverage` occasionally denies a correct, in-domain bundle. *Corroborated coverage* reverses such a denial when an *independent* signal—the mean BM25 relevance of the selected tools to the task—is high (≥ 4.0): a legitimate, domain-mis-routed bundle still has tools that lexically match the task, whereas a genuinely wrong-domain bundle does not. Over the model-independent validation profile, enabling the rescue cuts TRAC's unique over-denials of legitimate work from 233 to 212, at a cost of +0.08pp security-failure and four lost catches—a small Pareto refinement (the sweep knee is 4.0) that reintroduces no oracle.

5.4.3 Substrate Portability: Verified OPA/Rego Parity. The guarantee is a property of the *policy*, not our particular engine. PALADIN's RBAC, ABAC, and TRAC layers are authored as *rules-as-data*: the same YAML documents (`policies/{rbac,abac_rules,trac_rules}.yaml`) are evaluated both by our reference Python engine and, *unchanged*, by generic Rego policies on a standard Open Policy Agent runtime (OPA v1.18.0). Editing a rule updates both; no code is generated or hand-synced. We verify decision equivalence with the real `opa` binary: across 120 RBAC, 120 ABAC, and 100 unique-bundle TRAC evaluations, OPA reproduces *every* ALLOW/DENY decision and every advisory of the reference engine—zero mismatches. Parity holds by construction (both evaluate the identical rule data under the same operator semantics) and is confirmed empirically; the same policies also serve from a live `opa run -server` instance queried over OPA's REST Data API. The portable boundary is the rule-evaluation layer (S3/S4/S6, §3); the fact transformer (S5—BM25 domain inference and VerbNet action classification) runs once upstream and supplies both engines the same typed predicates. This is a *deployability* result (§3.5): the engine is a commodity, so the task-relational policy is enforceable on standard infrastructure—the artefact is the policy and its predicate, not a new runtime. Benchmarking the *performance* cost of the OPA substrate is the remaining follow-up (§6.4).

5.4.4 Dataset Ceiling and Post-Hoc Enforcement. Selection exact-match is intrinsically low (best-in-panel 9.4%, §5.2): tool-naming pedantry over 194 non-standardised identifiers, multiple defensible bundles scored against one gold, cross-persona ambiguity, and a brittle all-or-nothing match cap it independently of model capability. Conservatively the practical ceiling is 30–40%; Jaccard is the less-misleading signal.

Post-hoc enforcement. In selection mode the deterministic stack does not alter the LLM's selections: tool exact-match and Jaccard are recorded per row *before* policy gating, so the policy-gate security-failure differences are purely policy-attributable—PALADIN is a true post-hoc enforcement layer.

5.5 Planned Extensions

The following are *planned but not yet run*; we list them as concrete next experiments, each expanded in Future Work (§6.4).

Graded-difficulty ASTRA (1/2/3-tool). ASTRA scores single-tool gold bundles; we plan 2- and 3-tool variants to measure how the `capability_coverage` and `tool_relevance` predicates scale with bundle size, reported as a graded 1/3–3/3 recovery curve.

TOUCAN case study. A qualitative deployment on the TOUCAN agent–tool corpus to test whether the fixed thresholds transfer across benchmarks without re-tuning.

LLM-suggested policy refinement. Mine the model's free-text rationales for candidate TRAC rules, then audit and, if sound, promote them—an LLM-in-the-loop authoring loop that keeps enforcement deterministic.

User study. A practitioner study on the auditability of PALADIN's per-rule explanations versus a single opaque LLM verdict.

6 Discussion

6.1 Operating Point, Policy Dependence, and Generalisability

The floor's availability shortfall is an *inference* cost (§5.1.1): the valid requests it denies come almost entirely from the two BM25 predicates (`capability_coverage`, 160; `tool_relevance`, 52)—RBAC and ABAC deny no valid request by construction—and an *oracle* domain recovers most of it (~ 5pp router error, ~ 14pp genuine lexical sparsity, recoverable with a stronger retriever). Nor is the floor fixed: three knobs in `policies/trac_rules.yaml` (the `capability_coverage` floor, the `tool_relevance` cutoff, the rescue bar) and the choice of enforcing layers navigate the frontier (Fig. 2)—TRAC *advisory* at security-failure 22.3%, an over-provisioned RBAC (§5.3.3) at 19.7%—so we recommend advisory-TRAC as an availability-balanced default and the full floor where security is paramount (§6.4).

Generalisation across tool catalogs. With one fixed operating point (no per-domain tuning), across ASTRA's eight MCP domains (distinct tool catalogs) the model-independent floor's security-failure rate spans 0–21% by domain (`wikipedia`, a 10-tool catalog, the outlier)—not overfit to one distribution. Worst-group holds: in *every* catalog the floor's security-failure lies below *every* OpenAI generation's LLM-alone security-failure—per-catalog, not aggregate. So the fixed thresholds are robust *across catalogs within* ASTRA's construction; certifying the signal *beyond* it needs an independently-labelled or second corpus (§6.4).

The guarantee is conditional on the authored policy but bounded: removing any denial rule cannot decrease the security-failure rate (short-circuit), so the per-layer marginals (§5.3.3) upper-bound the damage from disabling a rule family.

6.2 The Floor's Adversarial Limit, Quantified

Our threat model (§3.2) assumes trusted task provenance—the operator authors the task; the agent proposes only the *bundle*—but we stress-test its violation, which few enforcement papers do. Giving an adversary the task text and appending the bundle's *own* tool names and descriptions (naive prompt-injection-style padding) collapses TRAC's catch on the 578 bundles it targets from 57.1% to 0.0%. Is this specific to the lexical BM25 predicates? We re-instantiate both enforcing predicates with a deterministic local semantic embedding (model2vec, no API): it catches *more* clean (57.1 → 70.9%) yet is *equally* defeated by padding (70.9 → 4.8%). The attack poisons the task text itself, so *no* predicate over that text—lexical or semantic—resists it: the evasion is a property of *untrusted task provenance*, and the control is input integrity (the composed prompt-injection layer, §3.2), not a stronger relevance function. Within the semi-honest model the guarantee holds; the embedding's higher clean-case catch is an option for deployments willing to trade determinism for a model dependency.

6.3 Limitations and Threats to Validity

Table 13 summarises the dominant threats to validity—internal, external, construct, and statistical—each with its mitigation and a pointer to the relevant section or released artefact.

6.4 Future Work and Artifact Availability

With full Anthropic and Google panels now reported (§5.3.1), the remaining cross-vendor extension is an **open-weights panel** (Llama, Mistral) beyond the three OpenAI generations and the two commercial non-OpenAI vendors. Computable from the released artefact without new LLM calls: a **Cedar port and OPA/Rego performance benchmark** (decision parity is already shown, §5.4.3; only latency/throughput cost is open), a **policy-perturbation sensitivity study** restricted to predicate-removal ablations, a **continuous threshold-grid sweep** over the three exposed TRAC knobs mapping the full security-failure/false-block surface. Requiring new LLM calls: a selection-mode LLM-on-LLM-bundles baseline (§5.3.3), **active adversarial inputs** beyond ASTRA's `wrong/null` model, a **second adversarial dataset** [10, 34, 49], **argument- and resource-instance-level authorisation** (beyond the tool-bundle granularity modelled here), and **multi-hop tool chains**.

We release the complete PALADIN implementation as open-source software.[1] The release includes the pipeline source; all policy configurations; 8 MCP server tool definitions (194 tools); the ASTRA v0.3 dataset; raw per-row logs for the reported runs (six

[1]Anonymous repository for review: https://anonymous.4open.science/r/PALADIN-anon; the original repository name describes the policy-stack components and will be renamed for the camera-ready.

Table 13: Threats to validity at a glance. Mitigations link back to the indicated section or released artefact.

Category	Threat & mitigation
Internal	
Pipeline ordering	Layer attribution conditioned on RBAC→ABAC→TRAC; per-layer marginals are ordering-independent (§5.3.3).
Implementation bugs	Raw per-row logs for every reported run and analysis scripts released; ASTRA replication cross-validates the verdict path against an external baseline (§5.2.2).
External	
Vendor coverage	Three OpenAI generations + a full blind non-OpenAI panel (Claude Opus 4.8, Claude Sonnet 4.6, Gemini 2.5 Pro; $n = 6{,}942$ each, §5.3.1); open-weights (Llama, Mistral) deferred (§6.4).
Dataset	ASTRA v0.3 is, to our knowledge, the only public corpus with the required label structure (§4.1); single-source risk is mitigated by fixed-threshold generalisation across its 8 tool catalogs (security-failure 0–21%, §6.1). A structurally comparable second corpus is future work.
Construct	
Per-bundle vs. per-tool	Our LLM-alone verdict is one LLM call per bundle; ASTRA LLM-ResM is per-tool with AND-aggregation—disclosed, consistent with the 0.075 F_1 gap direction (§5.2.2).
Positive-class choice	Model-alone tables score task resolution (positive = correct, persona-invariant); system tables score policy validity (positive = valid); each table states which (§4.5).
Adversarial task text	Task-text predicates (lexical *and* semantic—we test both, §6.2) are evaded by adversary-crafted task text; out of scope by construction (trusted provenance, §3.2), the defence is input integrity via composed prompt-injection controls [25].
Statistical	
Layer marginals	Paired task-level bootstrap 95% CIs (1,000 resamples); per-model and per-domain agreement provide robustness checks.
Policy dependence	Per-layer marginal security-failure upper-bounds damage of silently disabling a rule family; controlled perturbation study deferred (§6.4).
Domain inference	Task domain inferred from task text by BM25, consuming *no* ground-truth label (§4.6); an earlier `gt_mcps[0]` stand-in is removed, so both modes are leak-free. The ∼60–70% top-1 accuracy is the honest cost, mitigated by corroborated coverage (§5.4.2).

validation runs across three vendors and a selection baseline); analysis scripts regenerating every number; and a Dockerised runner with configurable ablations. (The released `policies/` retains legacy oracle-path files—e.g. `domain_capability_*.json`—not read by the evaluated agnostic pipeline.) *Citation hygiene:* recent or in-press references [6, 16, 21, 29, 30] are concurrent or near-concurrent work; none share authorship with this submission to the best of our knowledge.

7 Conclusion

Autonomous agents have turned the tool-invocation boundary into a security surface the Model Context Protocol leaves ungoverned, and the prevailing question—*is the model safe?*—is the wrong one. A model's unaided judgment is neither stable nor monotone in capability: across our panel, ungoverned tool admission ranges from a competent validator (gpt-4o, security-failure 26.8%) to a near-yes-machine that lets two-thirds of mis-scoped bundles through (gpt-5.4, security-failure 66.5%)—a 40pp security-failure span with no ordering by model recency. The operative question is what must be enforced *before* the call, regardless of which model authored the request—and our central result is that a thin *deterministic* admission layer answers it, holding the security-failure rate to a fixed 10.3% identically across every model. The guarantee is a property of the stack, not the LLM.

Our measurement framework is built to show this rather than assert it. By separating what the model contributes from what the policy contributes—through dual-mode evaluation under a single confusion-matrix convention—it makes the masking effect measurable, and in doing so surfaces structure prior task-tool benchmarks could not: three deterministic layers of differing marginal weight (RBAC and TRAC individually load-bearing, ABAC a backstop), first-firing attribution that overstates upstream necessity while hiding ABAC's necessity when RBAC is absent, and a retrieval intervention that improves accuracy and security together. PALADIN contributes a task-relational admission primitive within a security-first floor—not a complete new access-control model, nor an end-to-end agent-safety solution. Its lesson for the agentic era is a single design principle: authorisation must not ride on model trustworthiness. Determinism, not capability, is what turns an unpredictable model into a predictable security surface.

References

[1] Anthropic. 2024. Introducing the Model Context Protocol. Anthropic Blog. November 2024. https://www.anthropic.com/news/model-context-protocol

[2] Samuel Arcadinho, David Aparício, and Mariana S. C. Almeida. 2024. Automated Test Generation to Evaluate Tool-Augmented LLMs as Conversational AI Agents. Proceedings of the GenBench Workshop at EMNLP 2024. arXiv:2409.15934 [cs.CL] doi:10.48550/arXiv.2409.15934

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv preprint arXiv:2212.08073. https://arxiv.org/abs/2212.08073

[4] David Elliott Bell and Leonard J. LaPadula. 1973. *Secure Computer Systems: Mathematical Foundations.* Technical Report MTR-2547. MITRE Corporation.

[5] Ji-Won Byun, Elisa Bertino, and Ninghui Li. 2005. Purpose based access control of complex data for privacy protection. In *Proceedings of the 10th ACM Symposium on Access Control Models and Technologies (SACMAT).* Association for Computing Machinery, New York, NY, USA, 102–110.

[6] Kexin Chu. 2026. From Stateless Queries to Autonomous Actions: A Layered Security Framework for Agentic AI Systems. arXiv preprint arXiv:2604.23338. https://arxiv.org/abs/2604.23338

[7] Cisco Systems. 2025. Redefining Zero Trust in the Age of AI Agents and Agentic Workflows. Cisco Security Blog. Available: https://blogs.cisco.com/security/.

[8] Cloud Security Alliance. 2025. Fortifying the Agentic Web: A Unified Zero-Trust Architecture Against Logic-Layer Threats. CSA Blog. Available: https://cloudsecurityalliance.org/blog/2025/09/12/.

[9] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. Proceedings of the ACM on Programming Languages (OOPSLA). https://arxiv.org/abs/2403.04651

[10] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track. https://arxiv.org/abs/2406.13352

[11] Majed El Helou, Rahul Mehra, et al. 2025. ASTRA: A Dataset and Pipeline for Benchmarking Semantic Matching Between Tasks and Scopes for Delegated Authorization. arXiv preprint arXiv:2510.26702. https://arxiv.org/abs/2510.26702

[12] Majed El Helou, Benjamin Ryder, Chiara Troiani, Jean Diaconu, Hervé Muyal, and Marcelo Yannuzzi. 2026. Hybrid Inspection and Task-Based Access Control in Zero-Trust Agentic AI. arXiv preprint arXiv:2605.02682. https://arxiv.org/abs/2605.02682

[13] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. 2001. Proposed NIST Standard for Role-Based Access Control. *ACM Transactions on Information and Systems Security* 4, 3 (2001), 224–274. doi:10.1145/501978.501980

[14] Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. 2005. Verification and Change-Impact Analysis of Access-Control

Policies. In *Proceedings of the 27th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 196–205. doi:10.1145/1062455.1062502

[15] Philip W. L. Fong. 2011. Relationship-Based Access Control: Protection Model and Policy Language. In *Proceedings of the First ACM Conference on Data and Application Security and Privacy (CODASPY)*. Association for Computing Machinery, New York, NY, USA, 191–202.

[16] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv preprint arXiv:2503.23278. https://arxiv.org/abs/2503.23278

[17] Vincent C. Hu, David Ferraiolo, D. Richard Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations.* NIST Special Publication 800-162. National Institute of Standards and Technology.

[18] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermyn, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten, Marina Favaro, Jan Brauner, Holden Karnofsky, Paul Christiano, Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann, Ryan Greenblatt, Buck Shlegeris, Nicholas Schiefer, and Ethan Perez. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv preprint arXiv:2401.05566. https://arxiv.org/abs/2401.05566

[19] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv preprint arXiv:2312.06674.

[20] Invariant Labs. 2025. MCP Security Notification: Tool Poisoning Attacks. Technical Blog. Available: https://invariantlabs.ai/blog/.

[21] Saeid Jamshidi, Kawser Wazed Nafi, Arghavan Moradi Dakhel, Negar Shahabi, Foutse Khomh, and Naser Ezzati-Jivan. 2025. Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks. arXiv preprint arXiv:2512.06556. https://arxiv.org/abs/2512.06556

[22] Sandeep Kumar Jangam and Partha Sarathi Reddy Pedda Muntala. 2024. Comprehensive Defense-in-Depth Strategy for Enterprise Application Security. *International Journal of Multidisciplinary on Science and Management* 1, 3 (2024), 62–75.

[23] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv preprint arXiv:2503.15547. https://arxiv.org/abs/2503.15547

[24] Beth Levin. 1993. *English Verb Classes and Alternations: A Preliminary Investigation.* University of Chicago Press, Chicago, IL, USA.

[25] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. USENIX Security Symposium.

[26] Tayyab Muhammad et al. 2024. Integrative Cybersecurity: Merging Zero Trust, Layered Defense, and Global Standards for a Resilient Digital Future. ResearchGate Preprint.

[27] OASIS. 2013. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard.

[28] Jaehong Park and Ravi Sandhu. 2004. The UCON_ABC usage control model. *ACM Transactions on Information and System Security (TISSEC)* 7, 1 (2004), 128–174.

[29] KrishnaSaiReddy Patil. 2026. SentinelAgent: Intent-Verified Delegation Chains for Securing Federal Multi-Agent AI Systems. arXiv preprint arXiv:2604.02767. https://arxiv.org/abs/2604.02767

[30] Karthik Rampalli. 2026. PEDIGREE: Verifiable Delegation Identity for Agentic AI Systems. IETF Internet-Draft, draft-rampalli-pedigree-00. https://datatracker.ietf.org/doc/draft-rampalli-pedigree/00/

[31] Anand S. Rao and Michael P. Georgeff. 1995. BDI Agents: From Theory to Practice. In *Proceedings of the First International Conference on Multiagent Systems (ICMAS-95)*. The MIT Press, Cambridge, MA, USA, 312–319.

[32] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.

[33] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture.* NIST Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207

[34] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. International Conference on Learning Representations (ICLR). Spotlight. https://arxiv.org/abs/2309.15817

[35] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *IEEE Computer* 29, 2 (1996), 38–47. doi:10.1109/2.485845

[36] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/2302.04761

[37] Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, Ben Hagag, William L. Anderson, Peter Belcak, Ben Bucknall, Xiaohong Cai, Ayush Chopra, Doron Cohen, Ron F. Del Rosario, Andis Draguns, Annie Gray, Keren Katz, Vasilios Mavroudis, Jaron Mink, Sumeet Ramesh Motwani, Jonathan Petit, Leif-Sebastian Rembeck, Chandler Smith, John Sotiropoulos, Steven Young, Sarah Scheffler, and Mary Llewellyn. 2025. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents. arXiv preprint arXiv:2505.02077. https://arxiv.org/abs/2505.02077

[38] Karin Kipper Schuler. 2005. *VerbNet: A Broad-Coverage, Comprehensive Verb Lexicon.* Ph. D. Dissertation. University of Pennsylvania, Philadelphia, PA, USA.

[39] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. 2025. Authenticated Delegation and Authorized AI Agents. arXiv preprint arXiv:2501.09674. https://arxiv.org/abs/2501.09674

[40] SPIFFE Authors. 2022. SPIFFE: Secure Production Identity Framework for Everyone. CNCF Graduated Project, RFC 9542. https://spiffe.io/docs/latest/spiffe-specs/spiffe/

[41] Roshan K. Thomas and Ravi S. Sandhu. 1998. Task-Based Authorization Controls (TBAC): A Family of Models for Active and Enterprise-Oriented Authorization Management. In *Database Security XI: Status and Prospects*. Springer, Boston, MA, USA, 166–181.

[42] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291. https://arxiv.org/abs/2305.16291

[43] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science 18(6). https://arxiv.org/abs/2308.11432

[44] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155. https://arxiv.org/abs/2308.08155

[45] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. 2025. The Rise and Potential of Large Language Model Based Agents: A Survey. *Science China Information Sciences* 68, 2 (2025), 121101. https://arxiv.org/abs/2309.07864

[46] Tao Xiang, Zhexin Zhang, Yida Lu, Yuxin Liu, Jiaqi Song, and Minlie Huang. 2024. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. arXiv preprint arXiv:2406.09187. https://arxiv.org/abs/2406.09187

[47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2210.03629

[48] Tongxin Yuan, Zhiwei He, Lingpeng Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. Findings of the Association for Computational Linguistics: EMNLP 2024. https://arxiv.org/abs/2401.10019

[49] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of the Association for Computational Linguistics: ACL 2024. https://arxiv.org/abs/2403.02691

[50] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv preprint arXiv:2412.14470. https://arxiv.org/abs/2412.14470

[51] Yifan Zhu, Chao Zhang, Xin Shi, Xueqiao Zhang, Yi Yang, and Yawei Luo. 2025. MASTER: Multi-Agent Security Through Exploration of Roles and Topological Structures. arXiv preprint arXiv:2505.18572. https://arxiv.org/abs/2505.18572